429 が返ったら指数バックオフ＋ジッターで最大 5 回まで再試行するようにして。Retry-After があればそれを優先して

了解です。今の再試行の有無を確かめてから、4 つに分けて進めます。

retry_after.go を編集中 1:12

- ~~backoff の純関数を追加~~ 0:31
- Retry-After を秒数と日付の両方で読む 0:24
  - 「Retry-After」を検索 · 4 件 0.4s
  - client.go ほか 1 ファイルを読み込み 0.9s

  日付の形式で返るサーバーもあるので、`http.ParseTime` で両方受けます。上限は 30 秒にします。

  retry_after.go を編集中
- Do を包んで 429 と 503 を再試行
- テストを追加して通す

入力欄（現行のまま）

store パッケージのトランザクション境界を見直して、Repository ごとに Tx を受け取る形にして

考え中... 0:04

入力欄（現行のまま）

go.mod の Go を 1.25 に上げて、ビルドが通るか見て

ビルドが通るか確認中 0:18

タスク外の作業 ・ 3 件

go.mod を編集 0.2s

依存関係を整理 3.1s

ビルドが通るか確認中

入力欄（現行のまま）

README のセットアップ手順を、今の Makefile のターゲットに合わせて直して

Makefile を確かめてから README を直します。

完了 0:13

- ~~Makefile のターゲットを確認~~ 0:04
- ~~README の手順を書き換え~~ 0:07

`make setup` は無くなり、`make deps` と `make db-up` に分かれていたので、README の「はじめに」の手順を 3 行から 4 行に直しました。`make run` の説明は今のままで合っています。

ローカルの Postgres を Docker で立てる前提を、手順の先頭に 1 文足しています。

$0.04

入力欄（現行のまま）

auth.spec.ts のフレークを直して。ログイン後のリダイレクトで落ちることがある

中断 0:42

- ~~失敗するテストを 20 回まわして特定~~ 0:18
- storageState の書き込みを待つ 0:24
- CI で 3 回連続で通す

入力欄（現行のまま）

infra-terraform の plan を取って、差分を種類ごとに整理して

terraform plan -out=tfplan　無応答 1:30　中断 esc

plan を取る　1:52

terraform init -upgrade　6.2s

terraform plan -out=tfplan -lock-timeout=60s

差分を種類ごとにまとめる

入力欄（現行のまま）

429 が返ったら指数バックオフ＋ジッターで最大 5 回まで再試行するようにして。Retry-After があればそれを優先して

retry_after.go を編集中 1:20

タスク外の作業 ・ 2 件

~~backoff の純関数を追加~~ 0:31

思考 · 6 秒

backoff.go ほか 2 ファイルを編集 1.1s

ジッターは上限の半分から上限までの一様乱数にしました。`attempt` が大きいときの桁あふれも上限で止めます。

go vet で確認 1.4s

Retry-After を秒数と日付の両方で読む 0:32

「Retry-After」を検索 · 4 件 0.4s

retry_after.go を編集中

Do を包んで 429 と 503 を再試行

テストを追加して通す

入力欄（現行のまま）

429 が返ったら指数バックオフ＋ジッターで最大 5 回まで再試行するようにして。Retry-After があればそれを優先して

client_test.go を編集中 2:41

~~backoff の純関数を追加~~ 0:31

~~Retry-After を秒数と日付の両方で読む~~ 0:38

Do を包んで 429 と 503 を再試行 1:12

サブエージェント：呼び出し元の洗い出し · Explore · 完了 14s

client.go を編集 0.6s

internal/httpclient/client.go +9 −2 コピー

```
41  func (c *Client) Do(req *http.Request) (*http.Response, error) {
42          return c.hc.Do(req)
42          for attempt := 0; ; attempt++ {
43                  res, err := c.hc.Do(req)
44                  if !retryable(res, err) || attempt == maxRetries {
45                          return res, err
46                  }
47                  time.Sleep(wait(res, attempt))
48          }
49  }
```

再試行のテストを実行 4.8s

$ go test ./internal/httpclient/... -run TestRetry -count=1 exit 0

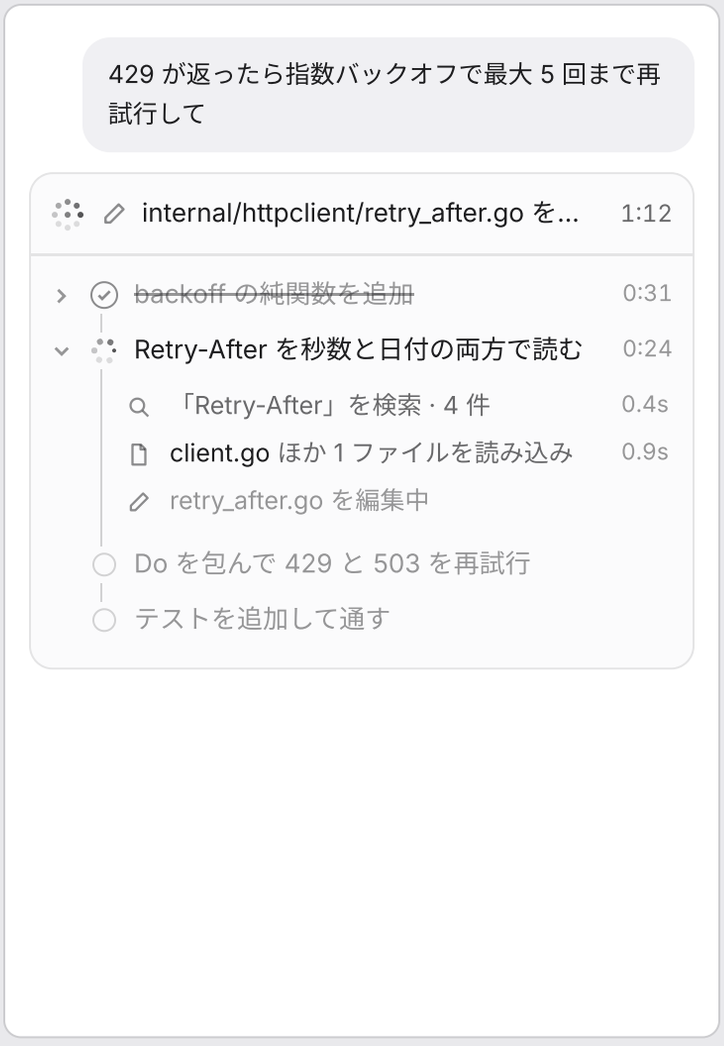
```
=== RUN   TestRetry429_RespectsRetryAfter
--- PASS: TestRetry429_RespectsRetryAfter (0.31s)
=== RUN   TestRetry503_Jitter
--- PASS: TestRetry503_Jitter (0.12s)
ok      ledger/internal/httpclient      1.204s
```

client_test.go を編集中

テストを追加して通す

入力欄（現行のまま）

グリッドの列 · 幅 360pt · ライト



グリッドの列 · 幅 360pt · ダーク · 完了



狭い列での決まり

見出しは 1 行で、今の作業の文言を末尾で省略。経過時間と無応答の表示は省略しない。

タスク名・作業の文言は 1 行で省略し、右端の所要時間は残す。

左右の余白を 14 → 10pt、作業の行の字下げを 40 → 36pt に詰める。

小さいタイルは会話を出さず、カードの見出しと同じ 1 行だけを下端に出す。無応答のときは中断ボタンを「esc」だけに縮める。

小さいタイルの状態の行（会話を出さないとき）



小さいタイルの状態の行 · ダーク · 無応答



見出し（1 行目）の全状態 · §4.2

状態	見出し	決まり
実行中 · タスク登録前	考え中… 0:09	中央は今の AgentActivityState の語（考え中・検索中・実行中・編集中・書いています・待っています）
実行中 · 作業あり	api.swift を編集中 0:13	今の作業の種類のアイコン＋作業の行と同じ文言（「〜中」）
応答なし	テストを実行中 **無応答 1:30** 中断 esc	経過の代わりに「無応答」と太字の時間。中断ボタンはこの状態だけ
承認・質問待ち	待機中… 0:47	点描は止めて薄くする。承認・質問そのものは入力欄の上（現行）
完了	完了 0:13	✓ は文字と同じ半透明グレー。右は所要時間
中断	中断 0:42	進行中だったタスクにも中断の印
エラー	エラー 0:21	エラーの本文はカードの中、起きた位置（FR-17）
次の入力へ続く	次の入力へ続く 0:36	実行中に次の入力を送ったとき。次の入力から新しいカード（FR-3）
終了	終了	終わり方の記録が無い（古い会話・異常終了）。印も時間も出さない

タスクの行 · §4.3



縦の線	印の中心を 1pt の線でつなぐ（スケッチの ○—○—○）。開いたタスクの作業の行の横も通す
開閉	左のシェブロン。進行中が完了したら自動で閉じ、次の進行中を開く。手で開いたものは自動で閉じない。作業が終わったら全部閉じる
所要時間	進行中になってから完了するまで。進行中は数え続ける。未着手は出さない
タスク外の作業	破線の丸。登録前の作業はタスクの上、それ以外は時系列の位置。所要時間の代わりに件数

作業の行 · §4.4



寸法と色

カード	角丸 12 · 1pt の線 · 面は会話の面より 1 段だけ明るい / 暗い
見出し	高さ 40 · 左右 14 · 点描 18 · アイコン 14 · 文字 13 · 時間 12（等幅数字）
タスクの行	高さ 28 · シェブロン 14 · 印 18（丸 12〜14）· 縦の線 1pt
作業の行	高さ 24 · 字下げ 40（狭い列 36）· アイコン 13 · 文字 12.5
色	アイコン・✓・印はすべて文字と同じ色の半透明（ライト 50% / ダーク 52%）。緑・状態の色・アクセントの色は使わない
時間の書き方	m:ss、1 時間以上は h:mm:ss。作業は 1.9s のように秒で

動き · VoiceOver

動くもの	見出しの点描と、進行中タスクの小さい点描だけ。作業の行にスピナーは出さない
動きを減らす	点描を止め、点を同じ濃さで並べた静止表示にする
見出し	「実行中。api.swift を編集中。経過 13 秒」と 1 つのラベルで読む
タスクの行	「完了。backoff の純関数を追加。31 秒。折りたたみ済み」
作業の行	「編集。client.go。0.6 秒。展開できます」

カードの外に置くもの

上	最初の作業・タスク登録より前の文章と思考（FR-16）
下	作業が終わった後の最後の返答とターンの使用量。終わった時点で一度だけ下へ移す（FR-18）
入力欄の上	承認待ちと未回答の質問（現行のまま · FR-19）